URL Shortener (like Bit.ly): System Design Notes

Table of Contents

0. What we're building
1. Requirements
2. Core entities
3. API design
4. High-level design
5. Deep dive: generating the short URL
6. Scaled architecture
7. Other points to raise in an interview
8. One-paragraph summary
9. ZooKeeper: coordinating the counter
10. Snowflake ID: no central counter at all
11. Comparing all ID generation approaches
12. Updated architecture

---

0. What we're building

A URL shortener takes a long URL and gives back a short one. When someone visits the short URL, they're redirected to the original.

```
https://www.facebook.com/anindya.s.dasgupta/  →  http://bit.ly/4kyLcpo
```

The design follows a standard interview flow: **Requirements → Entities → APIs → High-Level Design → Deep Dive**.

---

1. Requirements

Functional requirements (what the system does)

1. **Create a short URL from a long URL.**
   - *Optional:* support a **custom alias**. The user picks the short code, e.g. `bit.ly/my-portfolio`.
   - *Optional:* support an **expiration date**, after which the link stops working.
2. **Redirect.** Visiting the short URL sends the user to the original long URL.

Non-functional requirements (how well it does it)

Requirement	Detail	Why it matters
Low latency	~200 ms for both creation and redirect	Redirects sit in the user's click path, so slow redirects feel broken
Scale	100M daily active users, 1B total URLs	Drives storage, sharding, and caching decisions
Uniqueness	Every short URL must map to exactly one long URL	Two different long URLs must never share a code
CAP choice	Availability ≫ Consistency (eventual consistency is OK)	If a newly created link takes a second to work everywhere, that's fine. If the service is down, every link on the internet breaks.

Why availability over consistency? A redirect service is read-heavy and user-facing. A briefly stale read costs little, while downtime breaks every shared link at once.

Back-of-envelope estimates

These are assumptions to state out loud in an interview:

- **Read:write ratio.** ~80% reads and 20% writes, a 4:1 ratio. Real shorteners often run 100:1, so it's worth mentioning.
- **Writes.** Say 100M new URLs/day → 100M / 86,400 ≈ **~1,200 writes/sec**, with peaks around 3–5k/sec.
- **Reads.** At 4:1, that's ≈ **~5,000 reads/sec**, with peaks around 15–20k/sec.
- **Storage.** 1B URLs × ~500 bytes per record ≈ **~500 GB**. That's small enough for a handful of shards.

Short-code length math (Base62: a–z, A–Z, 0–9):

Length	Combinations
6 chars	$62^6 \approx$ **56.8 billion**
7 chars	$62^7 \approx$ **3.5 trillion**

Six characters already covers 1B URLs with lots of headroom. Seven is future-proof, which is why 6–7 characters is the target.

---

2. Core entities

- **short-url**: the generated code (e.g. `4kyLcpo`)
- **long-url**: the original destination
- **user**: who created the link (useful for custom URLs, analytics, and rate limiting)

---

3. API design

Create a short URL

```http
POST /v1/url
{
  "longURL": "https://www.facebook.com/anindya.s.dasgupta/",
  "customURL": "anindya",          // optional
  "expirationDate": "2027-01-01"   // optional
}
→ 201 Created
{ "shortURL": "http://bit.ly/4kyLcpo" }
```

Redirect

```http
GET /v1/url/{shortURL}
→ 302 Found
Location: https://www.facebook.com/anindya.s.dasgupta/
```

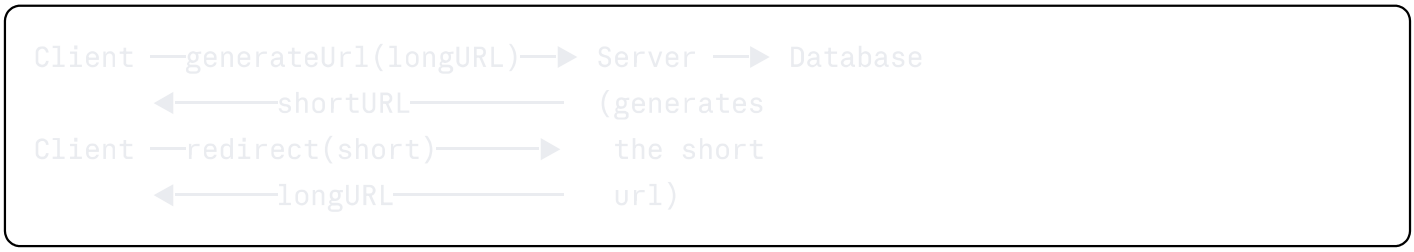
The redirect endpoint doesn't return JSON. It returns an HTTP redirect, and the choice of status code matters:

- **301 (Permanent).** The browser caches it, so later clicks never hit your server. This lowers load, but you lose click analytics and can't change or expire the link reliably.
- **302 (Temporary).** Every click hits your server. You can track analytics and enforce expiration. **Bit.ly-style services use 302.**

Errors to handle: `404` if the code doesn't exist, `410 Gone` if it has expired, and `409 Conflict` if a custom URL is already taken.

---

4. High-level design



- The **Client** sends two kinds of requests: *create* and *redirect*.
- The **Server** generates the short URL on create, and looks up the long URL on redirect.
- The **Database** stores the mappings.

Data model

URL table

Field	Notes
`shortURL`	**Primary key.** Every redirect looks up by this.
`longURL`	The destination
`customURL`	Set if the user chose an alias. It can simply *be* the shortURL, with a flag.
`expirationDate`	Nullable. Checked on redirect.
`userId`, `createdAt`	For ownership and analytics

User table: user metadata (id, name, email, plan, and so on).

Which database? The access pattern is a simple key-value lookup (`shortURL → longURL`) with no joins. That makes a **NoSQL key-value or wide-column store** (DynamoDB, Cassandra) a natural fit: easy horizontal sharding by `shortURL`, and high availability, which matches the CAP choice. A SQL database with sharding also works at 500 GB.

---

5. Deep dive: generating the short URL

There are two basic approaches.

Approach 1: Hashing



How it works:

1. SHA-1 produces a 160-bit hash (40 hex characters).
2. Take the first 6–7 characters, ideally after Base62-encoding the hash for more combinations per character.
3. Store the mapping and return it.

Pros:

- Deterministic. The same long URL always gives the same short URL, which deduplicates automatically.
- No coordination between servers is needed.

Cons:

- **Collisions.** Truncating a 160-bit hash to ~40 bits means two different URLs *will* eventually produce the same prefix. At 1B URLs, collisions are statistically certain (birthday paradox).
- **Collision handling costs a DB read on every write.** Check whether the code exists. If it does and maps to a *different* long URL, append a salt (e.g. `longURL + counter` or `longURL + timestamp`) and rehash. Repeat until the code is unique.
- The same URL gives the same code for every user, which breaks per-user analytics and per-user expiration unless you hash `longURL + userId`.

> **Terminology note:** the original whiteboard calls this "Encryption" and the lookup side "Decryption." Technically, hashing isn't encryption because it's one-way and can't be reversed. The "decryption server" doesn't decrypt anything; it **looks up** the long URL in the cache or database. In an interview, call these the **write/shortening service** and the **read/redirect service**.

Approach 2: Counter

```
Redis INCR counter → unique integer (e.g. 125,000,000,001) → Base62 encode →
"8M0kX1"
```

How it works:

1. Keep a global counter in **Redis**. The `INCR` command is atomic, so two servers can never get the same number.
2. Convert the integer to **Base62**, which yields a short string.
3. Store `shortURL → longURL` in the database.

Pros:

- **Zero collisions by design**, because every number is unique. No need to check the DB before writing.
- Fast: Redis `INCR` takes about **2–3 ms**.

Cons and fixes:

Problem	Fix
Redis becomes a single point of failure	Use Redis replication or Sentinel, or ZooKeeper for coordination (Section 9)
One network call per URL creation	**Range/batch allocation:** each server grabs a block of 1,000 IDs at once (e.g. server A gets 1–1000, server B gets 1001–2000) and hands them out from memory. The coordinator is hit once per 1,000 URLs.
Sequential codes are predictable (people can enumerate `...abc`, `...abd`)	Shuffle or obfuscate the number before encoding (e.g. bit-mixing or a reversible permutation), or add a random component
Lost IDs if a server crashes mid-range	Acceptable. With 3.5 trillion codes, wasted IDs don't matter.

Hashing vs. counter

	Hashing	Counter
Collisions	Yes, must check and retry	None
Write cost	Hash + DB check	Redis INCR (or in-memory with ranges)
Coordination	None	Needs a shared counter
Dedup of the same URL	Automatic	Needs a separate lookup if desired
Predictability	Random-looking	Sequential unless obfuscated

The counter with range allocation is usually the stronger interview answer.

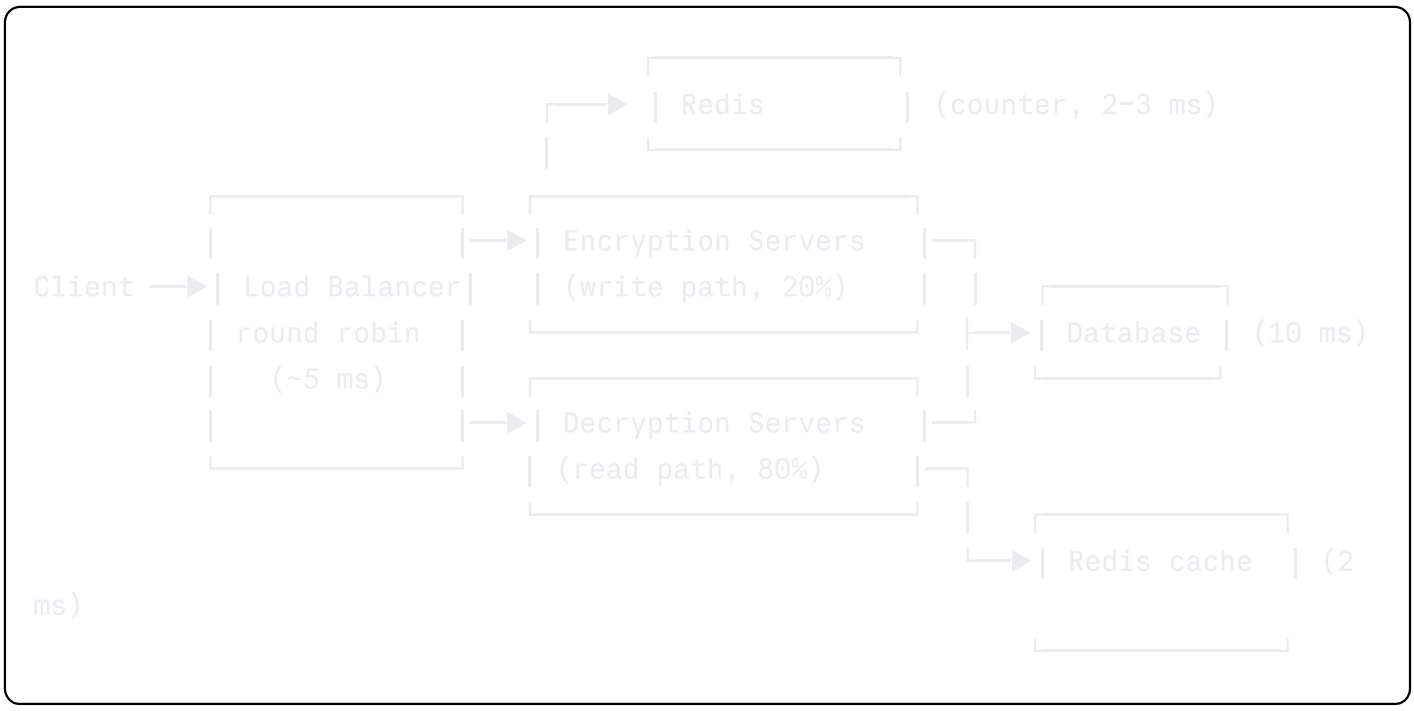
Custom URLs

Custom aliases skip generation entirely. The server checks whether the alias exists in the DB. If it's taken, return `409 Conflict`. If it's free, insert it. Use a conditional write (e.g. "insert if not exists") to avoid race conditions when two users claim the same alias at once. Custom aliases

must also never collide with counter-generated codes. You can use different lengths or character rules, or simply rely on the insert-if-not-exists check.

---

6. Scaled architecture



Components

Load balancer (round robin, ~5 ms)

- Distributes requests across servers evenly.
- Round robin works because the servers are **stateless**: any server can handle any request.
- In practice, it routes by path: `POST /v1/url` goes to encryption servers and `GET /v1/url/*` goes to decryption servers.

Encryption servers (write path, ~20% of traffic)

- Get the next ID from the Redis counter, Base62-encode it, and write to the DB.
- Fewer instances are needed because writes are the minority.

Decryption servers (read path, ~80% of traffic)

- Receive a short code, find the long URL, and return a 302 redirect.
- Check the **Redis cache first (~2 ms)**. On a miss, go to the **Database (~10 ms)**, then populate the cache.
- More instances are needed because this is the hot path.

Why split reads and writes into separate services? They have very different load profiles (80/20), so each can be **scaled independently**. A spike in redirects from a viral link shouldn't

slow down link creation, and vice versa.

Caching strategy (read path)

- **Cache-aside pattern:** read the cache → on a miss, read the DB → write to the cache → return.
- **Eviction policy: LRU.** Link popularity follows a power law, where a small fraction of links gets most of the clicks. Caching the hottest ~20% of links handles most traffic.
- **TTL:** set the cache TTL to no more than the link's expiration, so expired links aren't served from cache.
- Mappings almost never change once created, so the cache stays valid and hit rates are very high.

Latency budget (checking the 200 ms target)

Path	Hops	Approximate total
Redirect, cache hit	LB 5 ms + Redis 2 ms	**~7 ms**
Redirect, cache miss	LB 5 ms + Redis 2 ms + DB 10 ms	**~17 ms**
Create URL	LB 5 ms + Redis counter 2–3 ms + DB write 10 ms	**~18 ms**

All paths are well under 200 ms, even with network overhead. The requirement is met comfortably.

---

7. Other points to raise in an interview

- **Expiration cleanup.** Check `expirationDate` lazily on redirect (return 410). Also run a background job, or use the DB's native TTL (DynamoDB and Cassandra support this), to delete expired rows.
- **Database sharding.** Shard by `shortURL`, using hash-based partitioning or consistent hashing. Since every lookup is by `shortURL`, every query hits exactly one shard.
- **Replication.** Replicate the DB across nodes and regions for availability. Eventual consistency between replicas is acceptable, per the CAP decision.
- **Rate limiting.** Limit URL creation per user or IP to prevent abuse and spam, and to stop attackers from burning through the code space.
- **Security.** Scan long URLs against malware and phishing lists, since shorteners are commonly used to hide malicious links.
- **Analytics.** Don't write click counts to the DB synchronously on redirect, because that slows the hot path. Push click events to a queue (Kafka) and process them

asynchronously.

- **Global users.** Use a CDN or edge caching and multi-region deployment so redirects are served close to the user.

---

8. One-paragraph summary

A URL shortener is a **read-heavy key-value system.** On write, a service gets a unique ID from an **atomic counter** (batched into ranges for speed and resilience), **Base62-encodes** it into a 6–7 character code ($62^7 \approx 3.5$T combinations), and stores `shortURL → longURL` in a **horizontally sharded NoSQL database**. On read, a separately scaled redirect service checks a **Redis LRU cache** first, falls back to the DB, and returns a **302 redirect**. A round-robin load balancer spreads traffic across stateless servers. The system favors **availability over consistency**, and latency stays in the tens of milliseconds, far under the 200 ms target.

---

9. ZooKeeper: coordinating the counter

What ZooKeeper is

Apache ZooKeeper is a **distributed coordination service.** It doesn't store your application data. It stores small pieces of shared state that many servers need to agree on, such as config, locks, leader election, and counters.

Key concepts:

Concept	Meaning
Ensemble	A cluster of ZooKeeper nodes, usually 3 or 5. It keeps working as long as a **majority (quorum)** is alive, so 3 nodes tolerate 1 failure and 5 tolerate 2.
znode	A node in a file-system-like tree (e.g. `/shortener/counter`) that holds a small piece of data.
Persistent znode	Stays until explicitly deleted.
Ephemeral znode	Deleted automatically when the client that created it disconnects or dies. This makes it perfect for tracking live servers.
Sequential znode	ZooKeeper appends a monotonically increasing number to the name (e.g. `worker-0000000042`).
Version	Every znode has a version number. A write can say "only update if the version is still X." That's **compare-and-set (CAS)**, which gives atomic updates.
Watch	A client can subscribe to changes on a znode and gets notified when it changes.
ZAB protocol	ZooKeeper's consensus protocol. Every write goes through the leader and is agreed on by a quorum, so all clients see the same order of updates.

Why use it here

The Redis counter from Section 5 has two weaknesses: Redis can lose recent increments on failover (replication is asynchronous), and it's a single point of failure. ZooKeeper replicates every write to a quorum before confirming it, so **a range, once handed out, is never handed out again**, even if a node crashes.

How to use it: range allocation

The idea is that ZooKeeper hands out **blocks of IDs**, not individual IDs. Each write server grabs a range, serves IDs from memory, and only talks to ZooKeeper when the range runs out.

```
ZooKeeper: /shortener/counter = 3000

Server A claims 1-1000     → serves 1, 2, 3 … from memory
Server B claims 1001-2000  → serves 1001, 1002 … from memory
Server C claims 2001-3000  → serves 2001, 2002 … from memory

Server A runs out → claims 3001-4000 → counter = 4000
```

Step by step:

1. On startup, the server reads `/shortener/counter` along with its version (say value `3000`, version `7`).
2. It tries to write `4000` with the condition "only if version is still 7."
3. If that succeeds, the server owns IDs 3001–4000.
4. If it fails, another server claimed a range first. Re-read and retry.
5. The server hands out IDs from memory with **zero network calls** per URL.
6. When about 80% of the range is used, it **prefetches** the next range in the background so there's never a pause.

Code (Python, using the `kazoo` client):

```python
```

```python
from kazoo.client import KazooClient
import threading

RANGE_SIZE = 1000

class RangeAllocator:
    def __init__(self, zk_hosts="zk1:2181,zk2:2181,zk3:2181"):
        self.zk = KazooClient(hosts=zk_hosts)
        self.zk.start()
        self.counter = self.zk.Counter("/shortener/counter")  # CAS-based recipe
        self.lock = threading.Lock()
        self.current, self.end = self._claim_range()

    def _claim_range(self):
        self.counter += RANGE_SIZE            # atomic: retries on version conflict
        start = self.counter.pre_value + 1    # value before our increment
        end = self.counter.post_value         # value after our increment
        return start, end

    def next_id(self):
        with self.lock:
            if self.current > self.end:
                self.current, self.end = self._claim_range()
            id_ = self.current
            self.current += 1
            return id_
```

Then `short_code = base62_encode(allocator.next_id())` (see the Base62 function in Section 10).

Tracking live servers (optional)

Each server registers an **ephemeral znode** at `/shortener/servers/server-A`. If the server crashes, the znode disappears automatically, and other components can watch `/shortener/servers` to know who is alive. The unused part of a dead server's range is simply lost, which is fine with 3.5 trillion possible codes.

Trade-offs

Pros	Cons
Strongly consistent, so no duplicate ranges ever	Another cluster to run and monitor
Survives node failures (quorum)	Writes are slow-ish (tens of ms), but that's fine since it's hit once per 1,000 URLs
Per-URL cost is zero network calls	ZooKeeper is a **CP** system: it refuses writes if the quorum is lost

The CAP tension. Our system prefers availability, while ZooKeeper prefers consistency. Range allocation softens this: if ZooKeeper goes down, servers keep running on the IDs already in memory. With prefetching, a ZooKeeper outage only hurts if it lasts longer than a server takes to burn through its buffered IDs.

---

10. Snowflake ID: no central counter at all

What it is

Snowflake is an ID scheme created at **Twitter**. Every server generates unique IDs **locally**, with no coordination per ID, by packing a timestamp, a machine ID, and a sequence number into one 64-bit integer.

Bit layout (64 bits)

```
| 1 bit | 41 bits                | 10 bits    | 12 bits         |
| sign  | timestamp (ms)         | machine ID | sequence number |
|   0   | since a custom epoch   | 0–1023     | 0–4095          |
```

Part	Bits	What it gives
Sign	1	Always 0, keeping the number positive
Timestamp	41	Milliseconds since a custom epoch (e.g. Jan 1, 2026). 2^{41} ms ≈ **69 years** of IDs.
Machine ID	10	Up to **1,024** generator machines. Often split into 5 bits datacenter + 5 bits worker.
Sequence	12	Counter within the same millisecond on the same machine: **4,096 IDs per ms per machine** (~4 million/sec per machine)

Why it's unique: two IDs can only collide if they have the same millisecond, the same machine, and the same sequence number. That can't happen, because each machine increments its own sequence within a millisecond.

Bonus property: IDs are **roughly time-sorted**, because the timestamp occupies the high bits.

How generation works

1. Get the current time in ms.
2. If it's the **same ms** as the last ID, increment the sequence. If the sequence overflows past 4,095, **wait until the next ms**.
3. If it's a **new ms**, reset the sequence to 0.
4. If the clock **went backwards** (e.g. an NTP adjustment), refuse to generate or wait until the clock catches up. Otherwise you could produce duplicates.
5. Combine: `(timestamp << 22) | (machine_id << 12) | sequence`.

Code (Python):

```
python
```

```python
import time, threading

EPOCH = 1767225600000  # Jan 1, 2026 in ms

class Snowflake:
    def __init__(self, machine_id):
        assert 0 <= machine_id < 1024
        self.machine_id = machine_id
        self.sequence = 0
        self.last_ts = -1
        self.lock = threading.Lock()

    def _now(self):
        return int(time.time() * 1000)

    def next_id(self):
        with self.lock:
            ts = self._now()
            if ts < self.last_ts:
                raise Exception("Clock moved backwards, refusing to generate ID")
            if ts == self.last_ts:
                self.sequence = (self.sequence + 1) & 0xFFF    # 12 bits
                if self.sequence == 0:                         # overflow: wait for
                    while ts <= self.last_ts:
                        ts = self._now()
            else:
                self.sequence = 0
            self.last_ts = ts
            return ((ts - EPOCH) << 22) | (self.machine_id << 12) | self.sequence
```

Base62 encoding (used by both the counter and Snowflake approaches):

```python
```

```
ALPHABET = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

def base62_encode(n):
    if n == 0:
        return ALPHABET[0]
    out = []
    while n:
        n, r = divmod(n, 62)
        out.append(ALPHABET[r])
    return "".join(reversed(out))
```

Where ZooKeeper fits in: assigning machine IDs

Snowflake has one hard requirement: **no two live servers may share a machine ID.** Hardcoding IDs in config is fragile with autoscaling. The clean solution is to let ZooKeeper assign them with ephemeral znodes:

python

```
from kazoo.exceptions import NodeExistsError

def claim_machine_id(zk):
    for i in range(1024):
        try:
            zk.create(f"/shortener/workers/{i}", ephemeral=True, makepath=True)
            return i            # we own machine ID i
        except NodeExistsError:
            continue            # taken by a live server, try the next one
    raise Exception("No free machine IDs")

machine_id = claim_machine_id(zk)
generator = Snowflake(machine_id)
```

Because the znode is **ephemeral**, when a server dies its ID is released automatically and can be reused by a new server. ZooKeeper is touched **once at startup**, not per URL, so the ZooKeeper-availability concern mostly disappears.

The catch for a URL shortener: length

A 64-bit Snowflake ID in Base62 is up to **11 characters** ($62^{10} \approx 8.4\times10^{17} < 2^{63} \approx 9.2\times10^{18}$). That's noticeably longer than the 6–7 characters we wanted, and short links are the whole point of the product.

Ways to handle this:

- **Accept ~11 characters** if length isn't critical.
- **Use a compact Snowflake variant.** For example, use seconds instead of ms and fewer bits: 29 bits timestamp (~17 years) + 8 bits machine (256 servers) + 10 bits sequence (1,024 IDs/sec/server) = 47 bits ≈ **8 Base62 characters**. That supports ~260k writes/sec total, far above our ~5k/sec peak.
- **Use Snowflake for internal IDs** (e.g. the DB primary key or analytics events) and ZooKeeper range allocation for the public short code.

Trade-offs

Pros	Cons
No network call per ID, pure local computation	IDs are long (11 chars for standard 64-bit)
No central bottleneck or single point of failure	Depends on **clock sync** (NTP); clock drift backwards is dangerous
Scales linearly with machines	Machine IDs must be unique, which needs ZooKeeper or careful config
IDs are time-sortable	IDs leak creation time and are somewhat predictable

11. Comparing all ID generation approaches

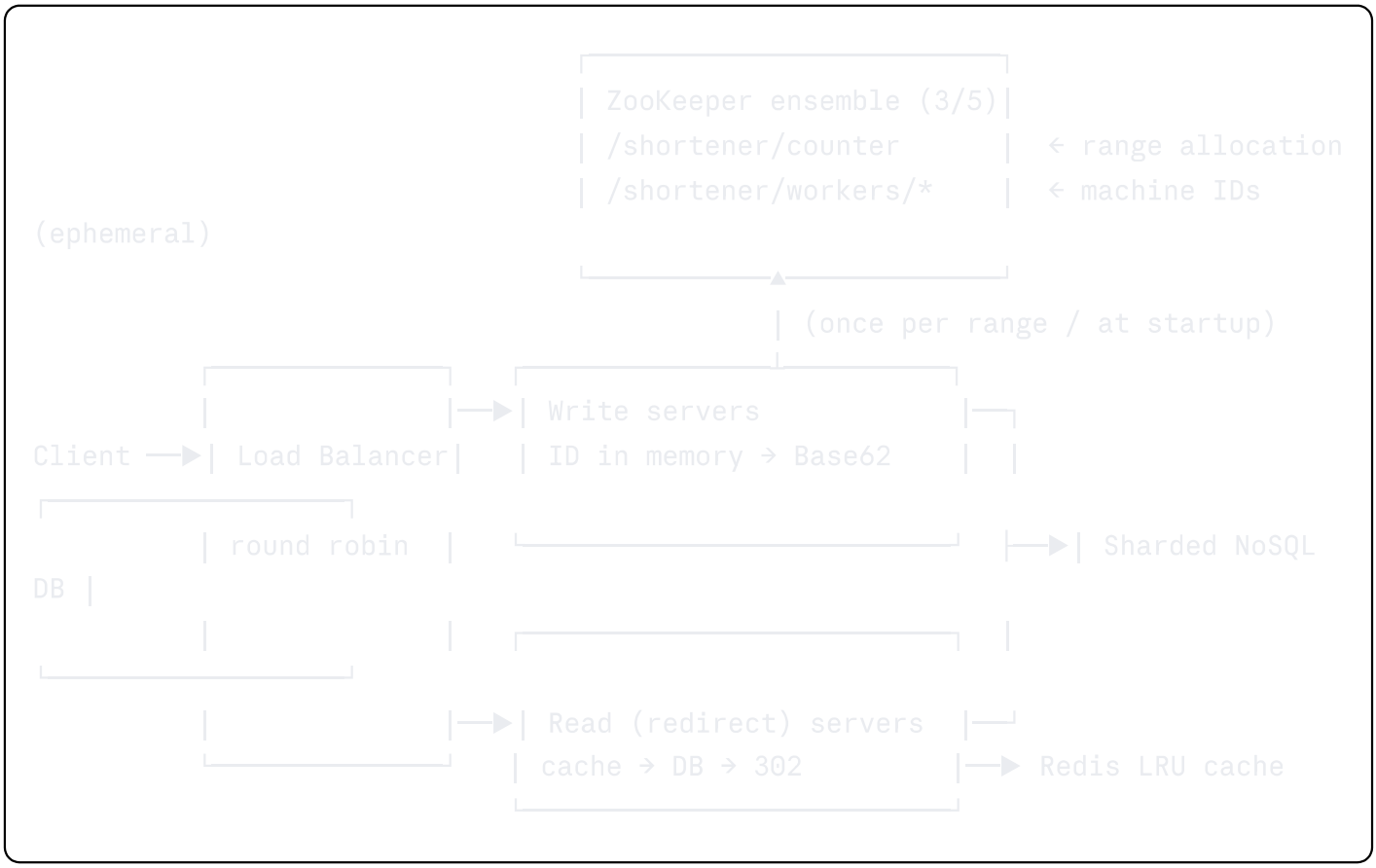
Approach	Per-ID network call	Collisions	Code length	Coordination	Main risk
Hash (SHA-1, truncated)	DB check on every write	Yes, needs retry	6–7	None	Collision handling cost
Redis counter	Yes (2–3 ms), unless batched	None	6–7	Redis	Redis SPOF, lost increments on failover
ZooKeeper range allocation	~0 (once per 1,000 IDs)	None	6–7	ZooKeeper	Ops overhead; CP during quorum loss
Snowflake	0	None	~11 (or ~8 compact)	ZooKeeper at startup only	Clock skew, longer codes

Interview recommendation:

- If **short codes** are the top priority, use **ZooKeeper range allocation + Base62.** It gives the shortest codes, no collisions, and nearly zero per-request overhead.
- If **zero coordination** and massive write scale matter more, use **Snowflake**, with ZooKeeper assigning machine IDs, and accept slightly longer codes.

---

12. Updated architecture



What changed from the original design: the Redis **counter** is replaced by ZooKeeper, either for range allocation or for Snowflake machine-ID assignment. Redis stays only as the **read cache**, which is where it's strongest. The write path now makes no coordination calls per request, so creating a URL costs only the LB hop plus the DB write, about **~15 ms**.